Scenario 2 Master Blueprint: Toast POS Offline-First Sync Engine

1. System Requirements & Context

Goal: Design an offline-first architecture for 500,000+ handheld Android POS terminals operating in unreliable Wi-Fi environments across 100,000+ restaurants, supporting 50M+ daily transactions.

- **Functional:** Local order creation, offline ticket printing, encrypted store-and-forward offline payment processing, idempotent multi-device state synchronization upon reconnection, and conflict resolution across multiple offline terminals.
- **Non-Functional:** 72-hour continuous offline operation budget, zero data loss on hardware crash/battery death, < 2s batch sync latency upon reconnection, and At-Most-Once payment processing (Zero double-charging guarantee).

2. Device Local Storage & Durability Strategy

To ensure physical device crashes or power loss do not corrupt un-synced orders, handhelds utilize an embedded SQLite database running in **Write-Ahead Logging (WAL)** mode.

- **WAL Sync Mode:** Set to PRAGMA synchronous = NORMAL with WAL enabled. Writes write to the WAL file sequentially before committing to main DB, providing atomic transaction guarantees with high write throughput.
- **Hardware Encryption (TEE):** Sensitive payment tokens are encrypted using hardware-backed Android KeyStore (Trusted Execution Environment) before hitting SQLite.

3. Event Sourcing & Device Data Schema

Instead of mutating order rows directly in place, the system uses an **Event Sourcing** model. Every POS action generates an immutable local event.

```
-- LOCAL EMBEDDED SQLITE SCHEMA (ON POS DEVICE)

TABLE Local_Event_Store {
  seq_id INTEGER PRIMARY KEY AUTOINCREMENT, -- Strict local sequence
  event_id UUID UNIQUE NOT NULL,            -- Global Idempotency Key
```

```
  check_id UUID NOT NULL,                -- Target Check/Order ID
  device_id UUID NOT NULL,               -- Originating Terminal ID
  event_type VARCHAR(50) NOT NULL,          -- e.g., 'ITEM_ADDED', 'ITEM_REMOVED',
'PAYMENT_CAPTURED'
  payload JSONB NOT NULL,                -- Action details (item_id, seat, timestamp)
  vector_clock JSONB NOT NULL,              -- e.g., {"dev_1": 4, "dev_2": 2}
  sync_status VARCHAR(20) DEFAULT 'PENDING'  -- 'PENDING', 'SYNCING',
'ACKNOWLEDGED'
}

TABLE Local_Offline_Payment {
  payment_id UUID PRIMARY KEY,
  check_id UUID NOT NULL,
  encrypted_card_token TEXT NOT NULL,       -- Encrypted via Gateway Public Key in TEE
  masked_pan VARCHAR(4) NOT NULL,           -- e.g. "4242"
  amount DECIMAL(10, 2) NOT NULL,
  offline_auth_code VARCHAR(50) NOT NULL,   -- Hardware-generated offline approval
  created_at TIMESTAMP NOT NULL,
  sync_status VARCHAR(20) DEFAULT 'PENDING'
}
```

4. Conflict Resolution Protocol: Operation-Based CRDTs + Vector Clocks

Scenario: Device A (offline) removes a $15 burger from Table 4. Device B (offline) adds a $10 dessert to Table 4. Both reconnect at the same time.

The Mechanism:

1. **Vector Clocks:** Every device tracks logical time vectors across terminals: VC = {DevA: 12, DevB: 8}.
2. **Commutative Operation Set:** Order line items are modeled as an Observed-Remove Set (OR-Set CRDT).
    - Adding an item creates a unique instance_id (e.g., Burger_UUID_99).
    - Removing an item appends a TOMBSTONE event targeting that specific instance_id.
3. **Deterministic Server Merge:** When event logs flush to the Cloud Sync Service, events are replayed in deterministic causal order. If Device A explicitly removed

Burger_UUID_99, the tombstone extinguishes the item regard\less of whether Device B added another item concurrently.

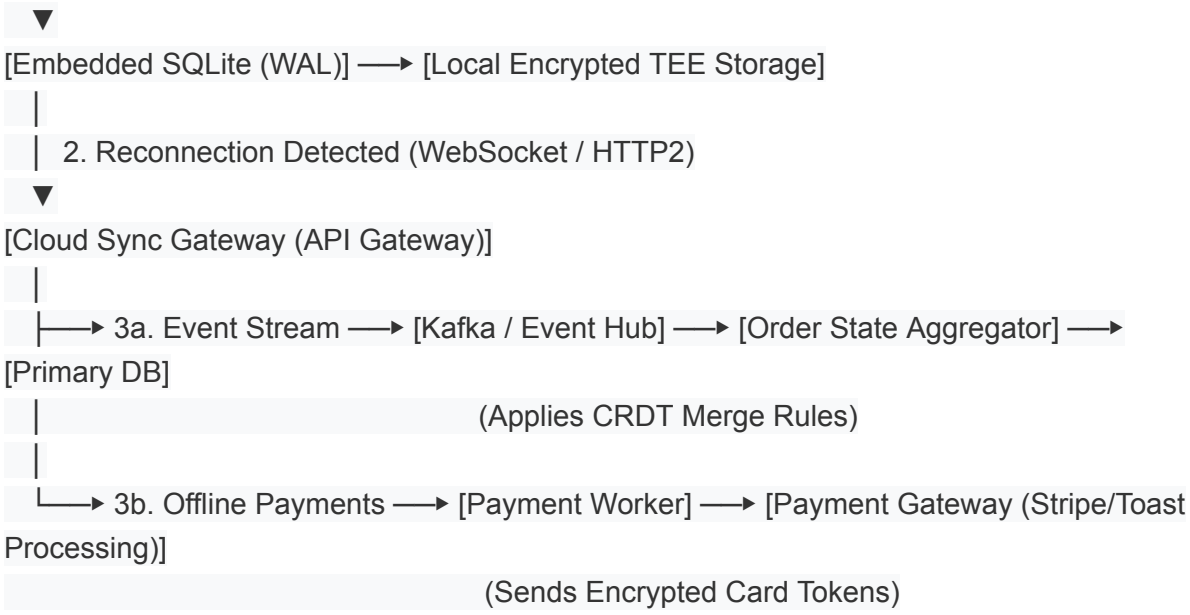
5. Offline Payment Processing & Store-and-Forward Risk Management

To process payments while disconnected without PCI-DSS compliance violations or catastrophic fraud loss:

Layer	Implementation	Risk Guarantee
Tokenization	Card swiped through EMV reader → encrypted inside Hardware Security Module (HSM/TEE) using Gateway Public Key.	Plaintext credit card numbers NEVER touch SQLite disk or RAM. PCI Scope is isolated to card reader.
Floor Limits	Terminals strictly enforce single transaction ceiling (e.g., $150 max) and total offline pool ceiling (e.g., $2,500 total).	Bounds total restaurant financial liability if offline cards are declined/stolen when network restores.
Store-and-Forward	Encrypted payloads queued in Local_Offline_Payment. On network reconnection, payments flush directly to Payment Gateway.	Guarantees At-Most-Once authorization through unique payment transaction idempotency tokens.

6. Architecture Sync Flow

[Android POS Terminal]
    │ 1. Action: Swipe Card / Modify Order



Section 7: Follow-Up Deep-Dive Q&A (Staff SE Panel Prep)

Question 1: PCI-DSS Compliance in Offline Payment Storage

Q: How do you store credit card transactions on an Android device for 72 hours without violating PCI-DSS regulations or exposing the merchant to data leaks if a handheld is stolen?
A:

1. **Asymmetric Key Encryption at Hardware Edge:** The POS application never possesses the private key required to decrypt credit card data. During payment reading (EMV chip/magnetic dip), the physical card reader hardware (MSR/SCR) encrypts the Primary Account Number (PAN) immediately using an asymmetric public key provided by the upstream Payment Gateway.
2. **Android KeyStore Execution:** Encrypted payment payloads are saved into SQLite wrapped in an envelope encrypted by the Android KeyStore Hardware-backed Trusted Execution Environment (TEE).
3. **Zero Local Plaintext Exposure:** If a thief roots the Android device or extracts the SQLite database file, they only obtain blob byte arrays encrypted with the Payment Gateway's public key. The merchant device cannot decrypt its own stored card data.

Question 2: Resolving Complex Multi-Device State Conflicts

Q: What happens if Server A (offline) voids a check, while Server B (offline) takes a $50 cash payment on the exact same check? How does the server resolve this split-brain scenario upon reconnection?

A:

1. **Business Domain Hierarchy over Pure LWW:** Pure Last-Write-Wins (LWW) is dangerous in financial domain models because it can discard money transactions. We apply deterministic domain precedence rules during event log reconciliation.
2. **Payment Superiority Rule:** A PAYMENT_CAPTURED event is an immutable financial record. When Server B's cash payment event is ingested, the system accepts the cash payment into the immutable ledger regardless of Server A's offline void request.
3. **Exception Escalation Workflow:** The state engine transitions the check into a REQUIRES_MANAGER_REVIEW flag state. The check remains open with the captured payment attached, and an alert is pushed to the manager's POS station highlighting the offline conflict (Void vs. Payment) for manual reconciliation.

Question 3: Idempotency & Network Flapping during Sync

Q: Network drops mid-way through flushing 200 offline events to the cloud. How do you guarantee zero duplicate orders or duplicate payments when the device reconnects and retries?

A:

1. **Client-Assigned UUID Event Keys:** Every event generated locally receives a cryptographically unique event_id (UUIDv4) at the moment of creation on the Android device.
2. **Server-Side Deduplication Table:** Incoming sync batches pass through an idempotent deduplication layer backed by Redis and PostgreSQL. The Cloud Sync Gateway checks: INSERT INTO Processed_Events (event_id, device_id) VALUES (...) ON CONFLICT (event_id) DO NOTHING;
3. **Transactional Outbox & Sequence Checkpoints:** The device sends events in strict local sequence batches (e.g., seq_id: 101 to 150). The cloud responds with a verified ack_sequence_id: 150. The device only marks events as SYNCED in local SQLite upon receiving explicit server ACK.

Question 4: Handling Declined Store-and-Forward Payments

Q: A server takes an offline credit card payment for $120. When the POS reconnects 12 hours later, the card authorization is declined (e.g., insufficient funds / stolen card). Who bears the risk, and how is it handled technically?

A:

1. **Contractual Floor Limits & Merchant Risk Acknowledgment:** Offline payments inherently carry financial authorization risk. Merchants configure risk parameters in Toast settings (e.g., "Allow maximum $100 offline per card").
2. **Automated Secondary Collection & Re-Drive:** If an offline card is declined during cloud batch processing, the Payment Engine initiates automated retry logic (1 to 2 re-drives over 24 hours for soft declines like network timeouts).
3. **Ledger Write-Off & POS Notification:** For hard declines (e.g., card cancelled/blocked), the system marks the check payment status as PAYMENT_FAILED_OFFLINE and emits an asynchronous notification to the restaurant manager's terminal, generating an accounting line-item write-off record in the Toast Financial Ledger.

Question 5: Android Crash Durability & SQLite Tuning

Q: How do you ensure that an Android terminal whose battery suddenly dies mid-order entry does not suffer SQLite database corruption or lost event writes?

A:

1. **SQLite WAL Mode with Explicit Transaction Boundaries:** SQLite is configured with PRAGMA journal_mode=WAL; and PRAGMA synchronous=FULL; for payment-critical commits. Writes append sequentially to the .wal file before acknowledging back to the application thread.
2. **Atomic Unit of Work Wrapper:** Order state updates write within explicit SQLite transactions:

```
db.beginTransaction();
try {
   insertEvent(event);
   updateLocalViewState(order);
   db.setTransactionSuccessful();
} finally {
   db.endTransaction();
}
```

3. **Startup Recovery Process:** When the Android device reboots, SQLite automatically scans the WAL file, rolling back incomplete transactions and replaying un-checkpointed WAL frames back into the main database file before accepting new UI inputs.

Question 6: P2P Local Mesh Synchronization without Internet

Q: If the internet goes down completely for 3 days, can POS handhelds inside the same

restaurant still sync order updates with each other over local Wi-Fi without cloud access?

A:

1. **Local Leader Election (mDNS / NSD):** Handhelds use Android Network Service Discovery (NSD) over local Wi-Fi to auto-discover peers. One terminal (or the local Toast Hub / Kitchen printer bridge) is elected as the **Local Mesh Leader** using a Raft-lite consensus algorithm.
2. **Local Peer Sync Protocol:** Terminals push local events directly to the Mesh Leader over TLS via local HTTP/2 or WebSockets.
3. **Cloud Flush on Reconnection:** The Mesh Leader maintains the local master event sequence. When WAN connectivity resumes, only the Mesh Leader flushes the consolidated event log to the cloud, eliminating thundering herd network congestion from dozens of handhelds syncing simultaneously.